Journal of Statistical Software

MMMMMM YYYY, Volume VV, Issue II. doi: 10.18637/jss.v000.i00

exdqlm: An R Package for Estimation and Analysis of Flexible Dynamic Quantile Linear Models

Raquel Barata
University of California Santa Cruz

Raquel Prado
University of California Santa Cruz

Bruno Sansó
University of California Santa Cruz

Abstract

We present the R package **exdqlm** as a tool for dynamic quantile regression. The main focus of the package is the estimation of flexible dynamic quantile linear models (exDQLMs). exDQLMs utilize a new family of error distributions for parametric quantile regression, the extended asymmetric Laplace (exAL), a generalization of the asymmetric Laplace (AL) which is commonly used in parametric quantile regression. Estimation of a dynamic quantile linear model which utilizes the AL is included in the package as a special case of our functions for estimation. Non-time-varying quantile regression models are also a special case of our methods. Further, routines for estimation of a nonlinear relationship between a quantile and response via a transfer function model are avaliable. The software is built with two seperate algorithms for inference; Markov chain Monte Carlo (MCMC) and importance sampling variational Bayes (ISVB). The ISVB algorithm enables fast inference of large time series at a fraction of the memory and computational costs of the MCMC. Several quantitative and visual diagnostics are avaliable in the package for evaluation the models. We illustrate the implementation of these functions and algorithms with a step-by-step guide on how to apply the **exdqlm** package via several analyses on real datasets.

Keywords: dynamic quantile regression, dynamic models, asymmetric Laplace, variational Bayes.

1. Introduction

The purpose of the R (R Core Team 2013) package **exdqlm** is to provide an accessible tool for

flexible Bayesian estimation of a single dynamic quantile. It provides the versatility to model any quantile regardless of the distributional shape of the data. The model form is flexible enough to incorportate relevant features into the evolution structure of the dynamic quantile. Computationally efficient methods enable model selection as well as fast posterior inference for large time-series datasets. Further, a straight-forward method for quantifying non-linear relationships between a response and input at a specified quantile is included in the software. The package is protected under the MIT License, and is avaliable from the Comprehensive R Archive Network (CRAN) at TBD.

The flexible dynamic quantile linear models (exDQLM) implemented in the package were first presented in Barata, Prado, and Sanso (2020). The exDQLM framework utilizes a recently developed family of error distributions for parametric quantile regression, the extended asymmetric Laplace (exAL; Yan and Kottas, 2017). The exAL has been shown to overcome many of the well-known shortcomings of the asymmetric Laplace (AL; Yu and Moyeed, 2001), a special case of the exAL and the most commonly used error distribution in Bayesian parametric quantile regression models. The form of the AL has several computational advantageous which attribute to its popularity. The exAL preserves these computational advantages, making it the ideal error distribution for computationally tractable, versitile dynamic quantile modeling.

Well-developed software implementations of non-time-varying quantile regression model which utilize the AL are avaliable in the Bayesian R package **bayesQR** (Benoit, Al-Hamzawi, Yu, and Van den Poel 2011), and a classical version which utilizes the check-loss (Koenker 2005) in **quantreg** (Koenker 2021). The methods used in the Bayesian package **bayesQR** are a special case of our exDQLM. There are many established packages for dynamic modeling including **dlm** (Petris and Gilks 2018) and **dynr** (Ou, Hunter, Chow, Ji, Chen, Hung, Lee, Li, and Park 2021), however, implementations of dynamic quantile modeling are much more limited. To our knowledge, the classical package **quantreg** includes a routine which allows for a time-evolving structure, but this is the extent of the software avaliable for dynamic quantile regression.

In the remainder of this paper we introduce the package as follows. In Section 2, we describe the modeling framework, including the algorithms used for efficient posterior inference, as well as our methods for including nonlinear relationships. In Section 3 we detail the model diagnostics implemented in the package. Then, in Section 4, we walk through three step-by-step examples on how to use the package. Finally, in Section 5, we present a summary of the package capabilities.

2. Extended dynamic quantile linear models

Consider a set of time-evolving responses, y_t, for times $t = 1, \ldots, T$. For each t, an extended dynamic quantile linear model (exDQLM) for inference on a single p_0-th quantile is defined by

$$\text{Observation equation:} \qquad y_t = \mathbf{F}_t' \boldsymbol{\theta}_t + \epsilon_t, \qquad \epsilon_t \sim \text{exAL}_{p_0}(0, \sigma, \gamma) \tag{1}$$

$$\text{System Equation:} \qquad \boldsymbol{\theta}_t = \mathbf{G}_t \boldsymbol{\theta}_{t-1} + \boldsymbol{\omega}_t, \qquad \boldsymbol{\omega}_t \sim \text{N}(0, \mathbf{W}_t). \tag{2}$$

Here $\mathbf{F}_t$ is the $q \times 1$ regression vector of the covariates corresponding to the $q \times 1$ parameter vector $\boldsymbol{\theta}_t$ at time t, $\mathbf{G}_t$ is the q-dimensional evolution matrix defining the structure of the

Input list	model				PriorSigma		PriorGamma		
Symbol	$F_{1:T}$	$G_{1:T}$	m_0	C_0	a_σ	b_σ	m_γ	s_γ	ν_γ
List component	FF	GG	m0	C0	a_sig	b_sig	m_gam	s_gam	df_gam

Table 1: Translation from mathematical symbols to parameters used **exdqlm** functions.

parameter vector evolution in time, and the normal distribution according to which the system error $\boldsymbol{\omega}_t$ evolves is q-variate. The observational errors ϵ_t of the exDQLM are distributed from an exAL with quantile p_0 fixed such that $\int_{-\infty}^{0} \text{exAL}_{p_0}(\epsilon_t|0,\sigma,\gamma)\text{d}\epsilon_t = p_0$. We denote this distribution as exAL_{p_0}.

The exAL is constructed from an extension of the location-scale mixture representation of the an AL (Kozumi and Kobayashi 2011), details of which can be found in Yan and Kottas (2017). This results in a mixture respresentation of $\text{exAL}_{p_0}(0,\sigma,\gamma)$ as follows:

$$\int\int_{\mathbb{R}^+\times\mathbb{R}^+} \text{N}(y_t|C(p,\gamma)\sigma|\gamma|s_t + A(p)v_t, \sigma B(p)v_t)\text{Exp}(v_t|\sigma)\text{N}^+(s_t|0,1)\text{d}v_t\text{d}s_t. \tag{3}$$

Here, $p = p(p_0,\gamma) = I(\gamma < 0) + \{[p_0 - I(\gamma < 0)]/g(\gamma)\}$ where $g(\gamma) = 2\Phi(-|\gamma|)\exp(\gamma^2/2)$ and $\Phi(\cdot)$ denotes the standard normal CDF. $A(p)$, $B(p)$, $C(p,\gamma)$ are the functions of p and γ; $A(p) = \frac{1-2p}{p(1-p)}$, $B(p) = \frac{2}{p(1-p)}$, $C(p,\gamma) = [I(\gamma > 0) - p]^{-1}$. Lastly, $\text{Exp}(v|\sigma)$ denotes the exponential distribution with mean σ and $\text{N}^+(s_t|0,1)$ denotes a normal distribution truncated to the positive reals with mean 0 and variance 1. This mixture representation of the exAL can be exploited to rewrite the exDQLM as the following hierarchical model for $t = 1,\ldots,T$:

$$y_t|\boldsymbol{\theta}_t,\sigma,\gamma,v_t,s_t \sim \text{N}(y_t|\mathbf{F}_t'\boldsymbol{\theta}_t + C(p,\gamma)\sigma|\gamma|s_t + A(p)v_t, \sigma B(p)v_t) \tag{4}$$

$$v_t, s_t|\sigma \sim \text{Exp}(v_t|\sigma)N^+(s_t|0,1) \tag{5}$$

$$\boldsymbol{\theta}_t|\boldsymbol{\theta}_{t-1},\mathbf{W}_t \sim \text{N}(\mathbf{G}_t\boldsymbol{\theta}_{t-1},\mathbf{W}_t). \tag{6}$$

Further details and methodology for the exDQLM can be found in Barata *et al.* (2020).

2.1. Prior specification

Under the Bayesian formulation, we specify priors for the initial state vector $\boldsymbol{\theta}_0$, scale parameter σ and skewness parameter γ. For $\boldsymbol{\theta}_0$, we apply a q-variate conjugate normal prior $\boldsymbol{\theta}_0 \sim \text{N}(\mathbf{m}_0,\mathbf{C}_0)$. For σ, we apply a conjugate inverse-gamma prior denoted $\text{IG}(a_\sigma,b_\sigma)$. The parameter γ has bounded support over the interval (L,U) where L is the negative root of $g(\gamma) = 1-p_0$ and U is the positive root of $g(\gamma) = p_0$ (Yan and Kottas 2017). Thus, we place a Student-t distribution truncated to the interval (L,U) as the prior for γ, i.e. $\gamma \sim \text{t}_{(L,U)}(m_\gamma,s_\gamma)$ with ν_γ degrees of freedom. Table 1 shows the prior parameters used in the **exdqlm** functions, and their corresponding mathematical symbols discussed in this section.

In contrast to the flat prior suggest by Yan and Kottas (2017), we find that using a proper prior on the skewness parameter promotes reliable posterior inference. Further, in many cases a strong prior on σ is necessary to guarantee fast posterior convergence and estimation. A more elaborate discussion on facilitating posterior convergence can be found in Barata *et al.* (2020), and examples of prior specification can be found in Section 4.

2.2. Posterior estimation

exdqlm function	`exdqlmMCMC`			`exdqlmISVB`		
Symbol	Σ_{MH}	N_{BURN}	N_{MCMC}	ϵ_{tol}	N_{IS}	N_{SAMP}
Input	`Sig.mh`	`n.burn`	`n.mcmc`	`tol`	`n.IS`	`n.samp`

Table 2: Translation from mathematical symbols to parameters used in functions `exdqlmMCMC` and `exdqlmISVB`.

The hierarchical representation of the exDQLM facilitates Bayesian posterior simulation using Markov Chain Monte Carlo (MCMC) algorithm, implemented in the function `exdqlmMCMC`. Gibbs sampling is used to update the sets of latent variables s_t and v_t for $t = 1, \ldots, T$. The dynamic regression coefficients can be sampled using a forward filtering backwards sampling algorithm. Finally, parameters γ and σ are updated jointly with a random-walk Metropolis-Hastings (MH) step on the rescaled logit- and log-scale, respectively. Joint acceptance is automatic in our MCMC routine, however the covariance matrix used to sample the parameters (Σ_{MH}) can be specified by the user and/or automatically computed from the burned sample. An example of this functionality can be found in Section 4.1. We perform a total of $N_{BURN} + N_{MCMC}$ MCMC iterations, discarding the first N_{BURN} resulting in N_{MCMC} posterior samples.

For purposes of model selection or large time series datasets, posterior sampling with the MCMC algorithm can be computationally intractable. To this end, we include in the package an importance sampling variational Bayes (ISVB) routine, `exdqlmISVB`, for fast and accurate approximate posterior estimation. The variational Bayes (VB) framework allows us to approximate the posterior distribution with partitioned variational distributions. For the exDQLM, these are recognizable closed-form distributions with the exception of the joint variational distribution of σ and γ. The variational distribution of σ and γ is approximated at each iteration using importance sampling (IS) with N_{IS} particles. The parameters of the remaining distributions are computed directly at each iteration, with the dynamic regression coeffients updated using a forward filtering backwards smoothing algorithm. After the algorithm reaches convergence with tolerance of ϵ_{tol}, N_{SAMP} samples are drawn from the variational distributions, resulting in an approximate sample of the posterior distribution.

Our comprehensive ISVB and MCMC algorithms as well as detailed VB methodology can be found in Barata *et al.* (2020). Table 2 shows the parameters used in the functions discussed in this section, as well as their mathematical symbols.

2.3. Evolution covariance specification via discount factors

Both estimation algorithms, `exdqlmMCMC` and `exdqlmISVB`, utilze discount factors to specify the time-evolving covariance matrices $\mathbf{W}_t$ introduced in Equation (2) (West, Harrison, and Migon 1985). More precisely, we define the evolution variance matrix $\mathbf{W}_t = \frac{1-\delta}{\delta}\mathbf{G}_t\mathbf{C}_{t-1}\mathbf{G}_t'$ where $\mathbf{C}_{t-1}$ denotes the posterior variance of the state vector $\boldsymbol{\theta}_{t-1}$ at time $t-1$ and δ is a discount factor such that $0 < \delta \leq 1$. The discount factor δ can be specified by the user via function parameter `df`. Selection of discount factors is typically done by optimizing some model checking criterion. We suggest using the Kullback-Leibler (KL) divergence of the one-step-ahead predictive distribution functions discussed in the following Section 3. A brief example of this method for discount factor selection can be found in Section 4.2.

Discounting by block of the state-space model is also a functionality of our estimation al-

Symbol	$\tilde{t}$	k	$\mathbf{F}_{(\tilde{t}+1):(\tilde{t}+k)}$	$\mathbf{G}_{(\tilde{t}+1):(\tilde{t}+k)}$
`exdqlmForecast` input	`start.t`	`k`	`fFF`	`fGG`

Table 3: Translation from mathematical symbols to additional parameters used in function `exdqlmForecast`.

gorithms. The function parameters `df` and `dim.df` allow users to specify a set of discount factors and their dimensions, respectively, within the state-space model. Examples of this feature can be found in Section 4. For a full review of discount factors including discounting by block, see West *et al.* (1985).

2.4. k-step-ahead forecast

For an arbitrary time $\tilde{t}$, the k-step-ahead future marginal distribution of the quantile is

$$\mathbf{F}'_{\tilde{t}+k}\boldsymbol{\theta}_{\tilde{t}+k}|y_1,\ldots,y_{\tilde{t}} \sim \mathrm{N}(\mathbf{F}'_{\tilde{t}+k}a_{\tilde{t}}(k), \mathbf{F}'_{\tilde{t}+k}\mathbf{R}_{\tilde{t}}(k)\mathbf{F}_{\tilde{t}+k}) \tag{7}$$

where $\mathbf{a}_{\tilde{t}}(k) = \mathbf{G}_{\tilde{t}+k}\mathbf{a}_{\tilde{t}}(k-1)$, $\mathbf{R}_{\tilde{t}}(k) = \mathbf{G}_{\tilde{t}+k}\mathbf{R}_{\tilde{t}}(k-1)\mathbf{G}'_{\tilde{t}+k} + \mathbf{W}_{\tilde{t}+k}$, $\mathbf{a}_{\tilde{t}}(0) = \mathbf{m}_{\tilde{t}}$, and $\mathbf{R}_{\tilde{t}}(0) = \mathbf{C}_{\tilde{t}}$, with $\mathbf{m}_{\tilde{t}}$ and $\mathbf{C}_{\tilde{t}}$ denoting the filtered mean and covariance of $\boldsymbol{\theta}_{\tilde{t}}$, respectively. This distribution is implemented in the function `exdqlmForecast`. Table 3 shows the parameters used in the function, as well as their mathematical symbols seen in this section.

2.5. Transfer function model

Quantifying the relationship between an input and a quantile of a response is a non-trivial task, particularly when the relationship is nonlinear. In mean-centric dynamic modeling, transfer functions are a straight-forward method to incorporate variables which measure the combined effect of current and past regression effects (West *et al.* 1985). Our transfer function extension to the exDQLM, presented in Barata *et al.* (2020), expands this methodology to the dynamic quantile regression setting as follows.

For $t = 1,\ldots,T$ and univariate input, X_t, a transfer function exDQLM with exponential decay is defined by

$$y_t|\boldsymbol{\theta}_t,\gamma,\sigma \sim \mathrm{exAL}_{p_0}(\mathbf{F}'_t\boldsymbol{\theta}_t + \zeta_t, \sigma, \gamma) \tag{8}$$

$$\boldsymbol{\theta}_t|\boldsymbol{\theta}_{t-1},\mathbf{W}_t \sim \mathrm{N}(\mathbf{G}_t\boldsymbol{\theta}_{t-1}, \mathbf{W}_t) \tag{9}$$

$$\zeta_t|\zeta_{t-1},\psi_{t-1},\omega_t \sim \mathrm{N}(\lambda\zeta_{t-1} + X_t\psi_{t-1}, \omega_t) \tag{10}$$

$$\psi_t|\psi_{t-1},\nu_t \sim \mathrm{N}(\psi_{t-1}, \nu_t). \tag{11}$$

Here ζ_t captures the effect of the current and past regression effects, and ψ_t determines the instantaneous effect X_t has on the quantile. The parameter λ is a quantity in the unit interval which represents the memory of the regression effect up to time t and decays at an exponential rate.

Conditional on a fixed λ, say $\tilde{\lambda}$, we augement the ISVB algorithm to incorporate the transfer function structure by simply replacing $\mathbf{F}_t, \boldsymbol{\theta}_t, \mathbf{G}_t$, and $\mathbf{W}_t$ in Equations (1)-(2) with $\tilde{\mathbf{F}}'_t = (\mathbf{F}'_t, 1, 0)$, $\tilde{\boldsymbol{\theta}}'_t = (\boldsymbol{\theta}'_t, \zeta_t, \psi_t)$, $\tilde{\mathbf{G}}_t = \mathrm{blockdiag}\left\{\mathbf{G}_t, \begin{pmatrix} \tilde{\lambda} & X_t \\ 0 & 1 \end{pmatrix}\right\}$, and $\tilde{\mathbf{W}}_t = \mathrm{blockdiag}\left\{\mathbf{W}_t, \begin{pmatrix} \omega_t & 0 \\ 0 & \nu_t \end{pmatrix}\right\}$, respectively. This augementation is implemented in the function `transfn_exdqlmISVB`, an extension of the routine found in `exdqlmISVB`.

Symbol	X_t	$\tilde{\lambda}$	$(\delta_\zeta, \delta_\psi)$	m_0^{tf}	C_0^{tf}
`transfn_exdqlmISVB` input	`X`	`lam`	`tf.df`	`tf.m0`	`tf.C0`

Table 4: Translation from mathematical symbols to additional parameters used in function `transfn_exdqlmISVB`.

Similar to the specification of $\mathbf{W_t}$ discussed in Section 2.3, discount factors $(\delta_\zeta, \delta_\psi)$ are utilized to specify $\begin{pmatrix} \omega_t & 0 \\ 0 & \nu_t \end{pmatrix}$. A bivariate normal conjugate prior on $(\zeta_0, \psi_0)' \sim N(m_0^{tf}, C_0^{tf})$ completes the transfer function model extension. Table 4 shows the additional parameters used in `transfn_exdqlmISVB`, as well as their mathematical symbols discussed in this section. Further details of the transfer function exDQLM can be found in Barata *et al.* (2020).

2.6. Special cases

As mentioned previously, non-time-varying quantile regression is a special case of our methodology. More specifically, when discount factors are set to 1, the evolution covariance $\mathbf{W}_t$ reduces to $\mathbf{0}$ resulting in non-time-varying parameters (West *et al.* 1985). Thus the methodology presented in Yan and Kottas (2017) which utlizes the exAL in a static model can also be implemented using our package with parameter `df = 1`.

The next special case of our exDQLM arises from the fact that the exAL is an extension of the AL (Yan and Kottas 2017). When $\gamma = 0$, the observational error ϵ_t in Equation (1) reduces such that $\epsilon_t \sim \text{exAL}_{p_0}(\epsilon_t|0, \sigma, \gamma) = \text{AL}_{p_0}(\epsilon_t|0, \sigma)$. This special case with observational errors distributed independedly from an AL is the dynamic quantile linear model (DQLM) presented in Gonçalves, Migon, and Bastos (2017). The DQLM can be impletemented with our package by fixing $\gamma = 0$ using the parameter `dqlm.ind = TRUE`.

The final special case is the intersection of the previous two. A static quantile regression model with observational errors distributed according to an AL is the well-known Bayesian parametric method for quantile regression first presented in Yu and Moyeed (2001). This methodology, utilized to create the Bayesian package for quantile regression **bayesQR**, can also be implemented with our package using parameters `df = 1` and `dqlm.ind = TRUE`.

3. Model Diagnostics

To evaluate the quantile inference and predictive performance of the exDQLM, several quantitative and visual model diagonstics are included in the package.

The one-step-ahead predictive distribution function introduced by Rosenblatt (1952) is a useful diagnostic for models in which the marginal forecast distributions are unrecognizable or difficult to compute (Huerta, Jiang, and Tanner 2003; Prado, Molina, and Huerta 2006). For the exDQLM, this distribution is defined as

$$u_t = \Pr(Y_t \leq y_t | y_{1:t-1}, v_{1:T}, s_{1:T}, \sigma, \gamma) \tag{12}$$

with $y_{1:t-1} = y_1, \ldots, y_{t-1}$, $v_{1:T} = (v_1, \ldots, v_T)$ and $s_{1:T} = (s_1, \ldots, s_T)$. Here u_t defines an independent sequence which is uniformly distributed on the interval $(0, 1)$ (Rosenblatt 1952). Conditional on $\{v_{1:T}, s_{1:T}, \sigma, \gamma\}$, the forecast distribution of y_t within the exDQLM is normally distributed. Therefore, conditional on MAP estimates $\{\hat{v}_{1:T}, \hat{s}_{1:T}, \hat{\sigma}, \hat{\gamma}\}$ we estimate the

sequence as

$$\hat{u}_t = \Phi(y_t|y_{1:t-1}, \hat{v}_{1:T}, \hat{s}_{1:T}, \hat{\sigma}, \hat{\gamma}) \tag{13}$$

where Φ denotes the normal CDF. Further details can be found in Barata *et al.* (2020).

Using this estimated sequence $\{\hat{u}_t\}$, we can assess the model performance in several ways. First, we visually examine the correlation of the estimated sequence via an ACF plot. Second, by transforming the values with a standard normal inverse CDF, we compare their distributional shape to that of a standard normal with a normal QQ-plot. To quantify the divergence of this transformed sequence from normality we compute the KL divergence (Kullback and Leibler 1951), $\mathrm{KL}(h, \phi) = \int_{-\infty}^{\infty} h(x) \log \frac{h(x)}{\phi(x)} \mathrm{d}x$, where h denotes the numerically approximated density of our transformed sample and ϕ is the standard normal density. A smaller KL divergence suggests a superior model fit. Lastly, the standardized forecast errors,

$$\frac{y_t - \mathsf{E}[y_t|y_{1:t-1}, \hat{v}_{1:T}, \hat{s}_{1:T}, \hat{\sigma}, \hat{\gamma}]}{\sqrt{\mathrm{Var}(y_t|y_{1:t-1}, \hat{v}_{1:T}, \hat{s}_{1:T}, \hat{\sigma}, \hat{\gamma})}} \tag{14}$$

can also be used as an additional graphical aid in examining the distribution at specific time points.

For a final diagnoitic, we include Gelfand and Ghosh (1998) posterior predictive loss criterion (pplc) with check loss function $\rho_{p_0}(x) = x[p_0 - I(x < 0)]$. That is,

$$\mathrm{pplc} = \sum_t \mathsf{E}[\rho_{p_0}(y_t^{obs} - y_t^{rep})|y_1, \ldots, y_T] \tag{15}$$

where the expectation is with respect to the posterior replicate distribution of y_t, $p(y_t^{rep}|y_1, \ldots, y_T)$. Again, a smaller pplc value suggest a superior model fit.

The model checking criteria discussed in this section are implemented in the function `exdqlmChecks`. Examples of their utility can be found in the following Section 4.

4. Examples

We begin by loading the **exdqlm** package used for all examples.

```
R> library(exdqlm)
```

4.1. Lake Huron

In this example, we consider the Lake Huron time series from the package **datasets**. The data are annual measurements of the level (ft) of Lake Huron from 1875 to 1972, shown in Figure 2. With this example we show how to: build a basic state-space structure, specify a prior on γ, run the MCMC algorithm, plot the estimated quantiles and forecast distributions.

To estimate the dynamic distribution of the data, we consider three quantiles, $p_0 = 0.95, 0.50,$ and 0.05. We begin by creating the state-space structure and prior used to estimate the quantiles (i.e. $\mathbf{F}_t, \mathbf{G}_t, m_0$, and C_0 discussed in Section 2). We choose to model the quantiles with a second order polynomial trend, which we construct with the `polytrendMod` function.

```
R> model = polytrendMod(order = 2, m0 = c(mean(LakeHuron),0), C0 = 10*diag(2))
R> model

$FF
[1] 1 0

$GG
     [,1] [,2]
[1,]    1    1
[2,]    0    1

$m0
[1] 579.0041   0.0000

$C0
     [,1] [,2]
[1,]   10    0
[2,]    0   10
```

To allow variability in the dynamic quantile, we select a discount factor of 0.9. Discount factors can also be optimized, an example of which we will show in Section 4.2. For this example, we place a point-mass prior on σ with `fix.sigma = TRUE` to facilitate convergence. A discussion on how to choose the location of the point-masses can be found in Section 4.2. Further, we place truncated student-t priors on the skewness parameters via `PriorGamma` as discussed in Section 2.1. Lastly, we limit the number of iterations with `n.burn = 700` and `n.mcmc = 300`. The R output is shown for only the last quantile.

```
R> M95 = exdqlmMCMC(y = LakeHuron, p0 = 0.95, model = model,
+                  df = 0.9, dim.df = 2,
+                  fix.sigma = TRUE, sig.init = 0.07,
+                  PriorGamma = list(m_gam = -1, s_gam = 0.1, df_gam = 1),
+                  n.burn = 700, n.mcmc = 300)
R> M50 = exdqlmMCMC(y = LakeHuron, p0 = 0.50, model = model,
+                   df = 0.9, dim.df = 2,
+                   fix.sigma = TRUE, sig.init = 0.4,
+                   PriorGamma = list(m_gam = 0, s_gam = 0.1, df_gam = 1),
+                   n.burn = 700, n.mcmc = 300)
R> M5 = exdqlmMCMC(y = LakeHuron, p0 = 0.05, model = model,
+                  df = 0.9, dim.df = 2,
+                  fix.sigma = TRUE, sig.init = 0.07,
+                  PriorGamma = list(m_gam = 1, s_gam = 0.1, df_gam = 1),
+                  n.burn = 700, n.mcmc = 300)

[1] "running isvb algorithm to initalize mcmc"
[1] "ISVB iteration 5: 2021-06-14 14:02:30"
[1] "ISVB iteration 10: 2021-06-14 14:02:31"
[1] "ISVB iteration 15: 2021-06-14 14:02:33"
```

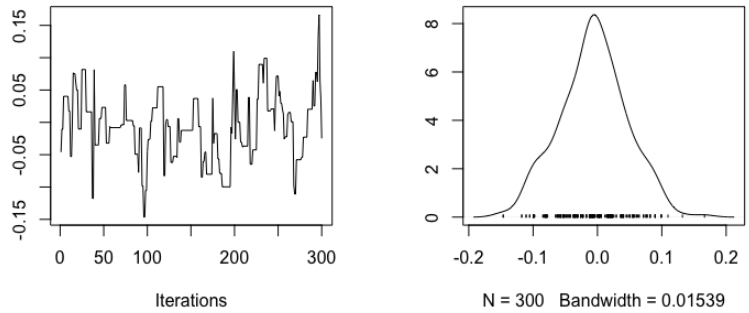

Figure 1: Example 1: Lake Huron. Trace plot (left) and density (right) of the MCMC samples of the skewness parameter γ estimated at the median.

```
[1] "ISVB converged: 15 iterations, 4.584 seconds"
[1] "burn-in iteration 500, acceptance rate 0.032: 2021-06-14 14:03:11"
[1] "MCMC iteration 1000, acceptance rate 0.127: 2021-06-14 14:03:52"
[1] "MCMC complete: 1000 iterations, 78.617 seconds"
```

By default `exdqlmMCMC` runs the ISVB algorithm to intialize the MCMC, which prints progress every 5 iterations. When the ISVB is used for initalization, the covariance matrix used to sample σ and γ in the MH step is automatically calculated from the ISVB samples of σ and γ. This covariance can be specified manually by using the setting `init.from.isvb = FALSE` and providing the covariance via input `mh_Sig`. Further, using the setting `joint.sample = TRUE`, the covariance matrix can be recalculated from burned samples.

After the ISVB algorithm converges, the `exdqlmMCMC` prints progress after each 500 MCMC iterations, along with the current acceptance rate of the MH step and system time. It is not uncommon for low acceptance rates to facilitate convergence, particularly for extreme quantiles (i.e. 0.05), as is seen in this example. The data are relatively symmetric, therefore we do not expect the skewness parameter γ to be far from zero for $p_0 = 0.5$. We can examine this with the samples from the posterior distribution of γ. All MCMC samples in the output of `exdqlmMCMC` are `mcmc` objects. Thus we can use the package **coda** to examine the trace plot and density of γ, seen in Figure 1.

```
R> library(coda)
R> traceplot(M50$samp.gamma, main = "")
R> densplot(M50$samp.gamma, main = "")
```

Clearly the γ is indistinct from zero, thus the added flexibility of the skewness parameter is not necessary for median. We re-run the model with a point-mass prior on γ at 0 using the settings `gam.init = 0` and `fix.gamma = 0`, equivalent to the setting `dqlm.ind = TRUE`.

```
R> M50 = exdqlmMCMC(y = LakeHuron, p0 = 0.50, model = model,
+                   df = 0.9, dim.df = 2,
+                   fix.sigma = TRUE, sig.init = 0.4,
```

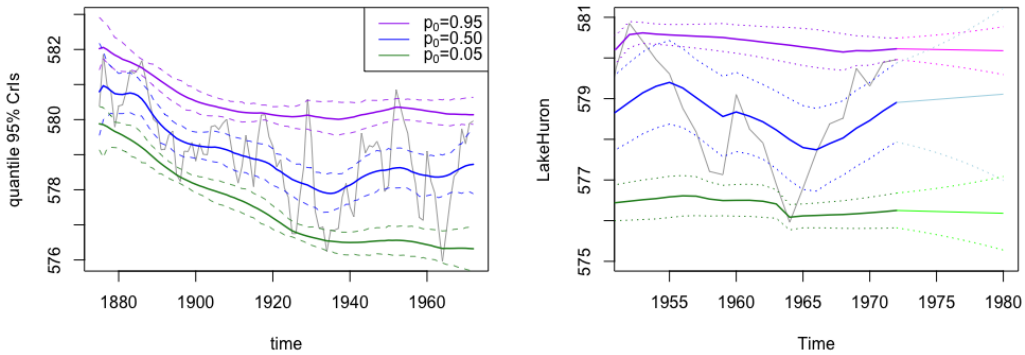

Figure 2: Example 1: Lake Huron. Left: MAP estimates and 95% CrIs of the estimated quantiles, plotted with the data (grey). Right: Forecasted quantile estimates and 95% credible intervals (seen after 1972), along with the filtered quantile estimates and 95% credible intervals (seen from 1952 to 1972).

```
+                    gam.init = 0, fix.gamma = TRUE,
+                    n.burn = 700, n.mcmc = 300)

[1] "running isvb algorithm to initalize mcmc"
[1] "ISVB iteration 5: 2021-06-14 14:14:54"
[1] "ISVB iteration 10: 2021-06-14 14:14:54"
[1] "ISVB iteration 15: 2021-06-14 14:14:54"
[1] "ISVB converged: 15 iterations, 0.479 seconds"
[1] "burn-in iteration 500, acceptance rate 0.998: 2021-06-14 14:15:37"
[1] "MCMC iteration 1000, acceptance rate 0.999: 2021-06-14 14:16:20"
[1] "MCMC complete: 1000 iterations, 86.013 seconds"
```

The results are `exdqlm` objects that can be used for analysis and forecasting. First we examine the estimated quantiles with the function `exdqlmPlot`, seen in Figure 2. This function generates a plot of the data with the MAP estimates and the 95% CrIs of the dynamic quantile. Subsequent quantiles (i.e. $p_0 = 0.50, 0.05$ in this example) can be added to the plot using the setting `add = TRUE`.

```
R> exdqlmPlot(y = LakeHuron, M95)
R> exdqlmPlot(y = LakeHuron, M50, add = TRUE, col = "blue")
R> exdqlmPlot(y = LakeHuron, M5, add = TRUE, col = "forest green")
R> legend("topright", lty = 1, col = c("purple","blue","forest green"),
+        legend = c(expression('p'[0]*'=0.95'),expression('p'[0]*'=0.50'),
+                   expression('p'[0]*'=0.05')))
```

Next, we forecast the $k = 8$ step-ahead distributions starting in 1972. To do this, we first create the observational vector (`fFF`) and evolution matrix (`fGG`) that will be used in the forecast updates. In this case both are non-time-varying and identical to those used to estimate the quantile.

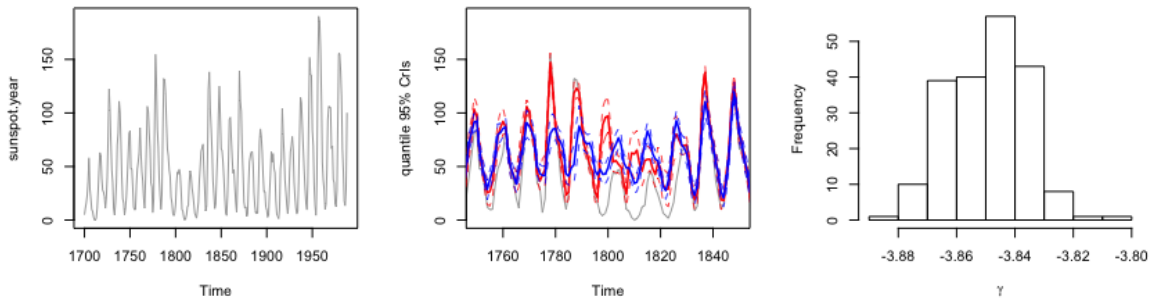

Figure 3: Example 2: Sunspots. Left: The sunspot time series from 1700 to 1988. Center: MAP estimates and 95% CrIs of the estimated quantiles from the DQLM (red) and exDQLM (blue), plotted with the data (grey) from 1750 to 1850. Right: Histogram of samples from the approximated posterior distribution of γ.

```
R> fFF = model$FF
R> fGG =  model$GG
```

We plot the time series data in a narrower range for closer examination and add the forecast estimates using `exdqlmForecast` with setting `add = TRUE`. Notice we start the forecast at the last time index of the data, i.e. `start.t = length(LakeHuron)`.

```
R> plot.ts(LakeHuron,  xlim = c(1952,1980), ylim = c(575,581),
+        col = "dark grey")
R> exdqlmForecast(y = LakeHuron, start.t = length(LakeHuron), k = 8, M95,
+               fFF = fFF, fGG = fGG, plot = TRUE, add = TRUE)
R> exdqlmForecast(y = LakeHuron, start.t = length(LakeHuron), k = 8, M50,
+               fFF = fFF, fGG = fGG, plot = TRUE, add = TRUE,
+               cols = c("blue","light blue"))
R> exdqlmForecast(y = LakeHuron, start.t = length(LakeHuron), k = 8, M5,
+               fFF = fFF, fGG = fGG, plot = TRUE, add = TRUE,
+               cols = c("forest green","green"))
```

This generates a plot of the data with the forecasted quantile estimates and 95% credible intervals (seen after 1972), along with the filtered quantile estimates and 95% credible intervals (before 1972), for reference. Results can be seen in Figure 2.

4.2. Sunspots

For our next example, we will use the yearly Sunspot time series from the package **datasets**. The data are yearly counts for sunspots from 1700 to 1988, shown in Figure 3. With this example we will show how to: use the **dlm** package to create the state-space model, combine blocks of a state-space model, apply the ISVB algorithm, choose a prior for σ to facilitate convergence, perform visual model diagnostics to compare the exDQLM with the DQLM, and use those diagnositcs for model selection.

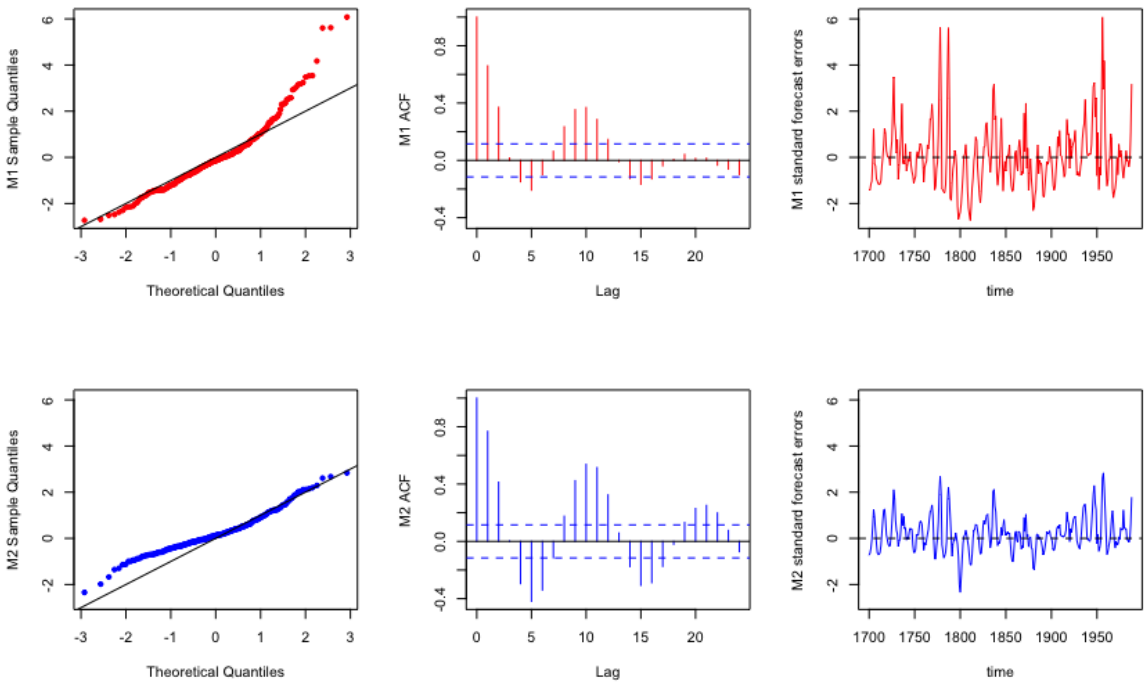

Figure 4: Example 2: Sunspots. The QQ-plots (left column), ACF plots (center column) and standard forecast errors (right column) from the DQLM (red) and exDQLM (blue).

The solar cycle impacts astronauts in space as well as lives and technology on earth. Of particular interest to scientists is the solar max, or period in which the sun is most active Fox (2012), therefore we will explore the 0.85 quantile in this example. Again, we begin by building the state-space structure used to estimate the quantile. Here we choose to model the quantile with a first order polynomial trend component as well as a fourier form seasonal component. The **exdqlm** functions are compatible with time-invariant `dlm` objects from the **dlm** package. For example, we create the first order trend component with the function `dlmModPoly` from the **dlm** package.

```
R> dlm.trend.comp = dlmModPoly(1, m0 = mean(sunspot.year), C0 = 10)
```

Next, we construct the seasonal component with our `seasMod` function. The duration of sunspot cycles typically follow a period of 11 years, of which we include the first 4 harmonics in the component.

```
R> seas.comp = seasMod(p = 11, h = 1:4, C0 = 10*diag(8))
```

To combine the two components into a single state-space structure, we use our function `combineMods`.

```
R> model = combineMods(dlm.trend.comp,seas.comp)

Warning message:
In combineMods(dlm.trend.comp, seas.comp) :
  m1 converted from a dlm object using 'dlmMod(m1)'
```

This produces a warning that the first input `dlm.trend.comp` was converted from a `dlm` object using our function `dlmMod`. This function can also be used directly on the `dlm` object to eliminate the warning. The resulting evolution matrix of the combined models is printed for illustration.

```
R> trend.comp = dlmMod(dlm.trend.comp)
R> model = combineMods(trend.comp,seas.comp)
R> model$GG

     [,1]    [,2]   [,3]    [,4]   [,5]    [,6]    [,7]    [,8]    [,9]
 [1,]    1  0.0000 0.0000  0.0000 0.0000  0.0000  0.0000  0.0000  0.0000
 [2,]    0  0.8413 0.5406  0.0000 0.0000  0.0000  0.0000  0.0000  0.0000
 [3,]    0 -0.5406 0.8413  0.0000 0.0000  0.0000  0.0000  0.0000  0.0000
 [4,]    0  0.0000 0.0000  0.4154 0.9096  0.0000  0.0000  0.0000  0.0000
 [5,]    0  0.0000 0.0000 -0.9096 0.4154  0.0000  0.0000  0.0000  0.0000
 [6,]    0  0.0000 0.0000  0.0000 0.0000 -0.1423  0.9898  0.0000  0.0000
 [7,]    0  0.0000 0.0000  0.0000 0.0000 -0.9898 -0.1423  0.0000  0.0000
 [8,]    0  0.0000 0.0000  0.0000 0.0000  0.0000  0.0000 -0.6549  0.7557
 [9,]    0  0.0000 0.0000  0.0000 0.0000  0.0000  0.0000 -0.7557 -0.6549
```

We will apply discount factors by component, i.e. a discount factor of 0.9 for the trend component of dimension 1 and a discount factor of 0.85 for the seasonal component of dimension

8. This discounting strategy can be applied with parameters `df = c(0.9,0.85)` and `dim.df = c(1,8)`.

Next we illustrate how to fix σ at a reasonable value. To do this, we investigate the scale of the data using the DQLM, a special case of the exDQLM described in Section 2.6. We call the function `exdqlmISVB` to apply the ISVB algorithm with `dqlm.ind = TRUE` and `fix.sigma = FALSE` to obtain estimates of the scale parameter σ. By default the ISVB algorithm prints progress every 5 iterations, seen below. This output is supressed for the remainder of this section.

```
R> M1 = exdqlmISVB(y = sunspot.year, p0 = 0.85, model = model,
+                  df = c(0.9,0.85), dim.df = c(1,8),
+                  dqlm.ind = TRUE, fix.sigma = FALSE)

[1] "ISVB iteration 5: 2021-06-14 06:51:04"
[1] "ISVB iteration 10: 2021-06-14 06:51:08"
[1] "ISVB iteration 15: 2021-06-14 06:51:11"
[1] "ISVB iteration 20: 2021-06-14 06:51:15"
[1] "ISVB iteration 25: 2021-06-14 06:51:19"
[1] "ISVB converged: 29 iterations, 23.461 seconds"
```

The σ estimates from the DQLM give a general idea of the scale of the data and are a good starting point for fixing σ (or setting a strong prior) in the exDQLM. For example, the MAP estimate of σ from the DQLM is approximately 4.

```
R> summary(M1$samp.sigma)

   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  3.470   3.806   3.935   3.934   4.054   4.513
```

Reducing this value, we place a point-mass prior on σ at 2. By default, `exdqlmISVB` places a point-mass prior on σ at the value of `sig.init`. We re-run the algorithm with `dqlm.ind = TRUE` to estimate the DQLM, as well as with the default settings to estimate the exDQLM.

```
R> M1 = exdqlmISVB(y = sunspot.year, p0 = 0.85, model = model,
+                  df = c(0.9,0.85), dim.df = c(1,8),
+                  dqlm.ind = TRUE, sig.init = 2)
R> M2 = exdqlmISVB(y = sunspot.year, p0 = 0.85, model = model,
+                  df = c(0.9,0.85), dim.df = c(1,8),
+                  sig.init = 2)
```

We can visualize the differences between the two resulting dynamic quantiles using the function `exdqlmPlot`, seen in Figure 3. For clarity we limit the figure to years 1750 to 1850.

```
R> plot(sunspot.year, xlim = c(1750,1850), col = "dark grey",
+       ylab = "quantile 95% CrIs")
R> exdqlmPlot(y = sunspot.year, M1, add = TRUE, col = "red")
R> exdqlmPlot(y = sunspot.year, M2, add = TRUE, col = "blue")
```

To examine whether the added flexibility of the exDQLM is significant, we can also visualize the approximate posterior distribution of the skewness parameter γ by plotting the samples from the corresponding variational distribution, also seen in Figure 3.

```
R> hist(M1$samp.gamma,xlab=expression(gamma),main="")
```

The approximate distribution is clearly distinct from zero, thus we expect a better model fit from the exDQLM.

To further examine the two models, we use the function `exdqlmChecks` to plot the model diagnostics discussed in Section 3. The results are seen in Figure 4. It is clear the QQ-plot and the scale of the standard forecast errors favor the exDQLM.

```
R> exdqlmChecks(y = sunspot.year, M1, M2, cols = c("red","blue"))
```

Finally, the function `exdqlmChecks` can also be used for model selection. Say we want to check whether the discount factor we used for the seasonal component was optimal with respect to the KL divergence of the one-step-shead predictive distributions. We can create list of possible discount factors, quickly run the ISVB for each possible model, and select the set of discount factors which results in the lowest KL divergence. A simple example is seen below.

```
R> possible.dfs = cbind(0.9,seq(0.85,1,0.05))
R> possible.dfs

     [,1] [,2]
[1,]  0.9 0.85
[2,]  0.9 0.90
[3,]  0.9 0.95
[4,]  0.9 1.00

R> KLs <- vector("numeric")
R> for(i in 1:nrow(possible.dfs)){
+   temp.M1 = exdqlmISVB(y = sunspot.year, p0 = 0.85, model = model,
+                   df = possible.dfs[i,], dim.df = c(1,8),
+                   sig.init = 2)
+   temp.check = exdqlmChecks(y = sunspot.year, temp.M1, plot = FALSE)
+   KLs = c(KLs,temp.check$m1.KL)
+ }
R> # optimal dfs based off KL divergence
R> possible.dfs[which.min(KLs),]

[1] 0.90 0.85
```

The results suggest the discount factor we used for the seasonal component is optimal.

4.3. Big Tree water flows

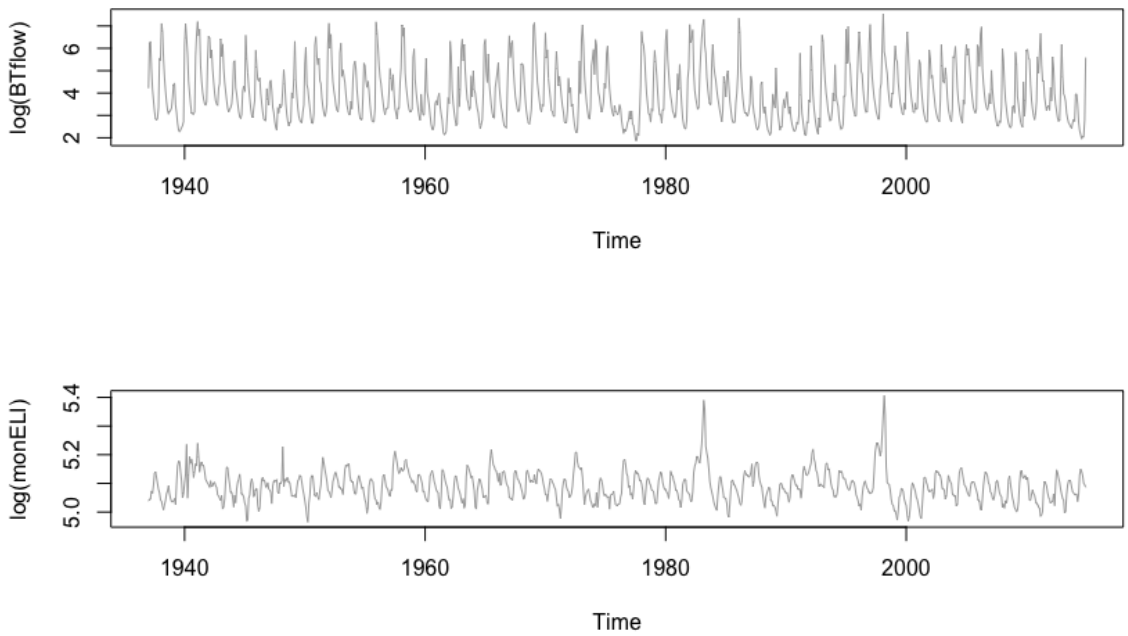

Figure 5: Example 3: Big Tree water flow. Top: Average monthy water flow at the Big Tree gauge of the San Lorenzo river in Santa Cruz County, CA from 1937 through 2014, plotted on the log-scale. Bottom: Average monthly ELI from 1937 through 2014.

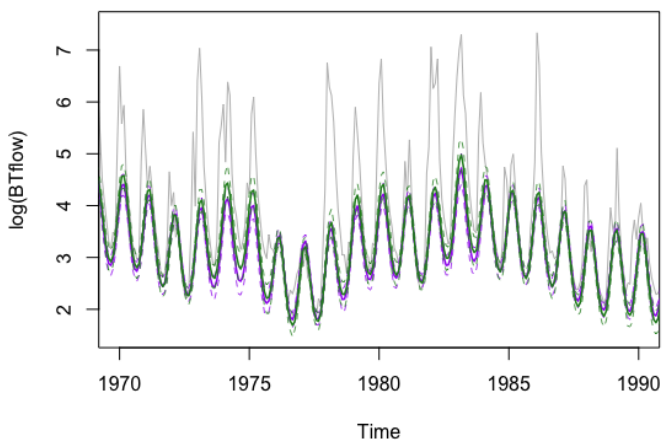

Figure 6: Example 3: Big Tree water flow. MAP estimates and 95% CrIs of the estimated quantiles from the models with the regression component (purple) and transfer function component (gree), plotted with the data (grey) from 1970 to 1990.

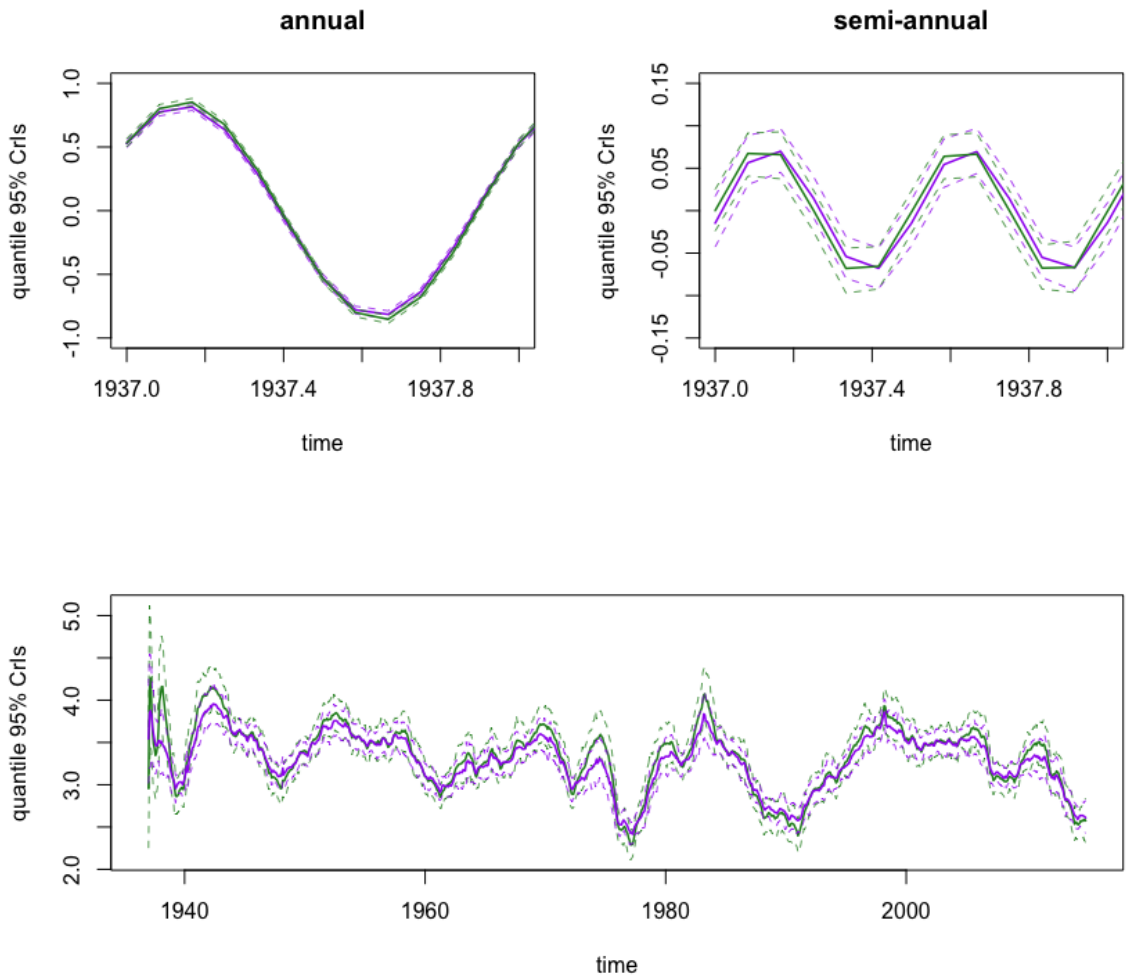

Figure 7: Example 3: Big Tree water flow. Top: MAP estimates and 95% CrIs of the annual (left) and semi-annual (right) components. Bottom: MAP estimates and 95% CrIs of the components which model the quantile association with ELI. The estimates from the dynamic quantile regression model are seen in purple, and estimates from the model with the transfer function components are in green.

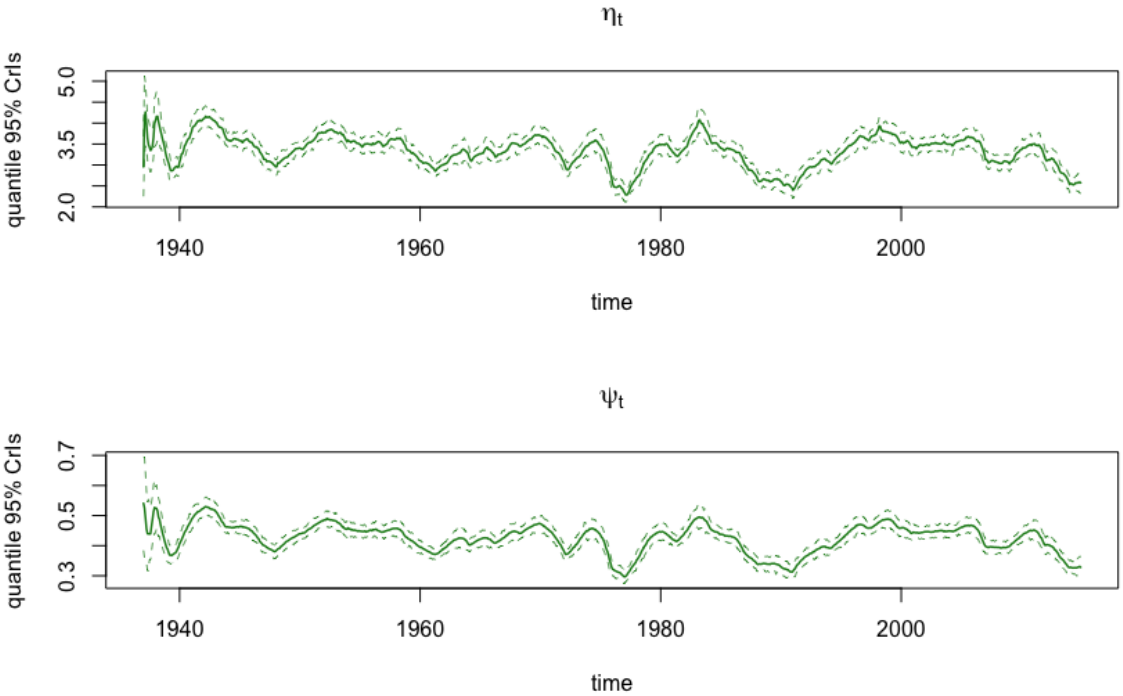

Figure 8: Example 3: Big Tree water flow. MAP estimates and 95% CrIs of parameters ζ_t and ψ_t from the transfer function component.

For our final example, we consider the average monthy natural water flow (cubic feet per second) at the Big Tree gauge of the San Lorenzo river in Santa Cruz County, CA from 1937 through 2014 (U.S. Geological Survey 2016). The dataset is included in our package as `BTflow`. With this example we show how to build a state-space structure manually (particularly for a dynamic quantile regression model), apply our transfer function exDQLM, examine indivdual components of the estimated quantiles, and perform quantitative model diagnostics to compare models.

For this particular dataset, the interest is in low quantiles as a possible indicator of drought, thus we will consider the $p_0 = 0.15$ quantile. To investigate any climatological influence on this low quantile, we include the recently developed climate index El Niño Southern Oscillation (ENSO) Longitude Index (ELI; Williams and Patricola, 2018) as in input of our exDQLM. This monthly ELI dataset is included in the package as `monELI`. We model the relationship between ELI and the quantile with two model structures: a dynamic quantile regression and a transfer function as seen in Section 2.5. To avoid modeling problems caused by the scale and extremes in the datasets, we fit the following models to the time series on the log scale seen in Figure 5.

To capture the seasonality seen in the water flow time series we include fourier form seasonal component in the model, again created with the `seasMod` function. For a period of twelve months, we include the first two harmonics (annual and semi-annual).

```
R> seas.comp = seasMod(p = 12, h = 1:2, C0 = diag(1,4))
R> seas.comp

$FF
[1] 1 0 1 0

$GG
           [,1]      [,2]       [,3]      [,4]
[1,]  0.8660254 0.5000000  0.0000000 0.0000000
[2,] -0.5000000 0.8660254  0.0000000 0.0000000
[3,]  0.0000000 0.0000000  0.5000000 0.8660254
[4,]  0.0000000 0.0000000 -0.8660254 0.5000000

$m0
[1] 0 0 0 0

$C0
     [,1] [,2] [,3] [,4]
[1,]    1    0    0    0
[2,]    0    1    0    0
[3,]    0    0    1    0
[4,]    0    0    0    1
```

Next, we create the dynamic regression component for our first model. A regression effect is easily written in a state-space format by setting the observational vector $F_t = X_t$ and $G_t = 1$, where X_t denotes the effect at time t. Thus we create this regression component for the state-space model manually as follows.

```
R> reg.comp <- NULL
R> reg.comp$m0 = 0
R> reg.comp$C0 = 1
R> reg.comp$FF = matrix(log(monELI),nrow = 1)
R> reg.comp$GG = 1
```

Combining this with the seasonal component we have our complete state-space for our dynamic regression model.

```
R> model.w.reg = combineMods(seas.comp,reg.comp)
```

To allow the ELI input components of both models to vary substaintially over time, we set the discount factors to be 0.85 (i.e. via the parameter `df` in the `exdqlmISVB` function and the parameter `tf.df` in the `transfn_exdqlmISVB` function). Alternatively for the seasonal component, we set the discount factor to 1 thus the seasonal components will not vary over time as discussed in Section 2.6. This will ensure any variation in the 0.15 quantile from year to year will solely be attributed to the effects of the ELI the input in our models.

The transfer function model is created within the function `transfn_exdqlmISVB` and follows the structure detailed in Section 2.5. Optimization using the KL divergence suggests the effects of ELI decay at an optimal transfer function rate of $\lambda = 0.35$.

```
R> possible.lams = seq(0.1,0.9,0.05)
R> KLs <- vector("numeric")
R> for(i in 1:length(possible.lams)){
+   temp.M1 = transfn_exdqlmISVB(y = log(BTflow), p0 = 0.15, model = seas.comp,
+                          X = log(monELI), tf.df = 0.85, lam = possible.lams[i],
+                          tf.m0 = rep(0,2), tf.C0 = diag(c(1,0.1),2),
+                          df = 1, dim.df = 4,
+                          sig.init = 0.1, gam.init = - 0.1,
+                          tol = 0.05)
+   temp.check = exdqlmChecks(y = log(BTflow), temp.M1, plot = FALSE)
+   KLs = c(KLs,temp.check$m1.KL)
+ }
R> # optimal lam based off KL divergence
R> possible.lams[which.min(KLs)]
```

```
[1] 0.35
```

With these discount factors and optimal λ, we estimate the dynamic quantile regression and transfer function models.

```
R> M1 = exdqlmISVB(y = log(BTflow), p0 = 0.15, model = model.w.reg,
+                  df = c(1,0.85), dim.df = c(4,1),
+                  sig.init = 0.1, gam.init = - 0.1,
+                  tol = 0.05)
R> M2 = transfn_exdqlmISVB(y = log(BTflow), p0 = 0.15, model = seas.comp,
```

```
+                          X = log(monELI), tf.df = 0.85, lam = 0.35,
+                          tf.m0 = rep(0,2), tf.C0 = diag(c(1,0.1),2),
+                          df = 1, dim.df = 4,
+                          sig.init = 0.1, gam.init = - 0.1,
+                          tol = 0.05)
```

To visualize the differences between the two resulting dynamic quantiles we use the function `exdqlmPlot`, results are found in Figure 6. For clarity we limit the figure to view 1970 to 1990.

```
R> plot(log(BTflow), col = "grey", xlim = c(1970,1990), ylim = c(1.5,7.5))
R> exdqlmPlot(y = log(BTflow), M1, add = TRUE)
R> exdqlmPlot(y = log(BTflow), M2, add = TRUE, col = "forest green")
```

Clearly the resulting quantiles have significant differences. To identify from where these differences arise, we can utilize the function `compPlot` to examine the components of the models more closely. We examine the annual and semi-annual components of the parameter vector first. The annual component is comprised of the first two elements of the parameter vector $\boldsymbol{\theta}_t$, thus we set the parameter `index = c(1,2)` to visualize this component. Similarly for the semi-annual component, the parameter `index = c(3,4)` as this component is the third and fourth elements of the vector. Both are non-time-varying so we examine only the first 12 time points, seen in Figure 7.

```
R> plot(NA,xlim = c(1937,1938), ylim = c(-1,1), xlab = "time",
+      ylab = "quantile 95% CrIs", main = "annual")
R> compPlot(y = log(BTflow), M1, index = c(1,2), add = TRUE)
R> compPlot(y = log(BTflow), M2, index = c(1,2), col = "forest green", add = TRUE)
R> plot(NA,xlim = c(1937,1938), ylim = c(-0.15,0.15), xlab = "time",
+      ylab = "quantile 95% CrIs", main = "semi-annual")
R> compPlot(y = log(BTflow), M1, index = c(3,4), add = TRUE)
R> compPlot(y = log(BTflow), M2, index = c(3,4), col = "forest green", add = TRUE)
```

The seasonal components of the two models are similar, thus we expect the differences to appear in the ELI component of each model. Again, we use the function `compPlot` to visualize these components seen in Figure 7.

```
R> compPlot(y = log(BTflow), M2, index = c(5,6), col = "forest green")
R> compPlot(y = log(BTflow), M1, index = c(5), add = TRUE)
```

For the transfer function model, `M2`, it is also of interest to look at the estimates of parameters ζ_t and ψ_t directly. Because these parameters augemented the state-space model as discussed in Section 2.5, the function `compPlot` can also be used to look at these estimates directly using the parameter `just.theta = TRUE`. This setting returns the dynamic posterior estimates of a single index of the state vector θ (or $\tilde{\theta}$ in the case of the transfer function model). Parameter `index` of 5 and 6 correspond to ζ_t and ψ_t, respectively. Results can be found in Figure 8

```
R> compPlot(y = log(BTflow), M2, index = 5, col= "forest green", add = FALSE, just.theta =
R> title(expression(eta[t]))
```

	M1		M2	
	output	value	output	value
pplc	`m1.pplc`	694.31	`m2.pplc`	589.07
KL	`m1.kl`	0.0144	`m2.kl`	0.0148

Table 5: Example 3: Big Tree water flow. Model diagnostic output from `exdqlmChecks` and resulting values.

```
R> compPlot(y = log(BTflow), M2, index = 6, col = "forest green", add = FALSE, just.theta
R> title(expression(psi[t]))
```

Lastly, we use the function `exdalmChecks` to compute the KL divergence and pplc discussed in Section 3, results of which are seen in Table 5.

```
R> checks.out = exdqlmChecks(y = log(BTflow), M1, M2,
+                            col = c("purple","orange"), plot = FALSE)
```

The KL divergences are very similar, however the pplc clearly favors the nonlinear relationship captured by the transfer function model.

5. Conclusion

Our exDQLM framework provides a powerful and flexible tool for dynamic quantile modeling. Relevant features of the data can easily be incorporated in the evolution of the quantile. The MCMC algorithm provides a method for exact posterior inference. Alternatively, the ISVB algorithm provides a method for fast approximate inference enabling estimation for large datasets, quick examination relevant model features and model selection. Finally, our transfer function model provides a straight-forward method for quantifying a nonlinear relationship between a quantile and response. These capabilities have been discussed in detail and demonstrated with a set of examples. The **exdqlm** package makes these methods accessible to all levels of users.

References

Barata R, Prado R, Sanso B (2020). "Fast inference for time-varying quantiles via flexible dynamic models with application to the characterization of atmospheric ." *Technical report*, University of California Santa Cruz, Department of Statistical Science. URL https://www.soe.ucsc.edu/research/technical-reports/UCSC-SOE-20-14.

Benoit D, Al-Hamzawi R, Yu K, Van den Poel D (2011). ***bayesQR**: A Bayesian approach to quantile regression*. R package version 2.3, URL https://CRAN.R-project.org/package=bayesQR.

Fox K (2012). "Solar Minimum; Solar Maximum." URL https://www.nasa.gov/mission_pages/sunearth/news/solarmin-max.html.

Gelfand AE, Ghosh SK (1998). "Model choice: a minimum posterior predictive loss approach." *Biometrika*, **85**(1), 1–11.

Gonçalves K, Migon HS, Bastos LS (2017). "Dynamic quantile linear model: a Bayesian approach." *arXiv preprint arXiv:1711.00162*.

Huerta G, Jiang W, Tanner MA (2003). "Time series modeling via hierarchical mixtures." *Statistica Sinica*, pp. 1097–1118.

Koenker R (2005). "Quantile regression."

Koenker R (2021). ***quantreg**: Quantile Regression*. R package version 5.85, URL https://CRAN.R-project.org/package=quantreg.

Kozumi H, Kobayashi G (2011). "Gibbs sampling methods for Bayesian quantile regression." *Journal of statistical computation and simulation*, **81**(11), 1565–1578.

Kullback S, Leibler RA (1951). "On information and sufficiency." *The annals of mathematical statistics*, **22**(1), 79–86.

Ou L, Hunter M, Chow SM, Ji L, Chen M, Hung HJ, Lee J, Li Y, Park J (2021). ***dynr**: Dynamic Models with Regime-Switching*. R package version 0.1.16-2, URL https://CRAN.R-project.org/package=dynr.

Petris G, Gilks W (2018). ***dlm**: Bayesian and Likelihood Analysis of Dynamic Linear Models*. R package version 1.1-5, URL https://CRAN.R-project.org/package=dlm.

Prado R, Molina F, Huerta G (2006). "Multivariate time series modeling and classification via hierarchical VAR mixtures." *Computational Statistics & Data Analysis*, **51**(3), 1445–1462.

R Core Team (2013). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria. URL http://www.R-project.org/.

Rosenblatt M (1952). "Remarks on a multivariate transformation." *The annals of mathematical statistics*, **23**(3), 470–472.

US Geological Survey (2016). "National Water Information System data available on the World Wide Web (USGS Water Data for the Nation)." URL http://waterdata.usgs.gov/nwis/.

West M, Harrison PJ, Migon HS (1985). "Dynamic generalized linear models and Bayesian forecasting." *Journal of the American Statistical Association*, **80**(389), 73–83.

Williams IN, Patricola CM (2018). "Diversity of ENSO events unified by convective threshold sea surface temperature: a nonlinear ENSO index." *Geophysical Research Letters*, **45**(17), 9236–9244.

Yan Y, Kottas A (2017). "A new family of error distributions for Bayesian quantile regression." *arXiv preprint arXiv:1701.05666*.

Yu K, Moyeed RA (2001). "Bayesian quantile regression." *Statistics & Probability Letters*, **54**(4), 437–447.

Affiliation:

Raquel Barata
PhD Candidate, Department of Statistics
University of California Santa Cruz
Santa Cruz, CA 95064
E-mail: rbarata@ucsc.edu

Journal of Statistical Software http://www.jstatsoft.org/
published by the Foundation for Open Access Statistics http://www.foastat.org/
MMMMMM YYYY, Volume VV, Issue II *Submitted:* yyyy-mm-dd
doi:10.18637/jss.v000.i00 *Accepted:* yyyy-mm-dd